PROCEEDINGS VOLUME OF THE 7TH IET INTERNATIONAL SMART CITIES SYMPOSIUM,
7TH SCS23, 3-5 DECEMBER 2023, UNIVERSITY OF BAHRAIN, BAHRAIN

Effective Selection of Completely Fair Scheduler Algorithm in RAID Kernel for Improved I/O Performance Using Machine Learning

P. K. Dutta[1], Satya Vir Singh[2], and Arup Nandi[3]

[1]Department of Electronics and Communication Engineering, Amity University Kolkata Kolkata, India[1]
[2]Vice Rector, Sharda University Uzbekistan Andijan, Uzbekistan[2]
[3]IT-Infrastructure Amity University -IT Kolkata, India[3]
E-mail: pkdutta@kol.amity.edu[1]

Keywords: RAID Prediction, XGBoost, Random Forest, Decision Tree, Heterogeneous Domains, RAID Levels, Web Ap- plication.

Abstract

The widespread adoption of cloud-based RAID (Re-dundant Array of Independent Disks) technologies is a resultof the rapid growth in data and the increasing need for high-performance storage. This paper aims to investigate the working model of a completely fair scheduler algorithm in cloud-based RAID systems that uses a machine learning-based approach to select the most suitable RAID level, optimizing I/O performance. We provide an overview of several proposed techniques such as file classification and separation, deployment of files on multiple SSDs, machine learning for RAID performance measurementand I/O isolation scheme. //To prevent kernel threads from blocking user-thread parallelism drop-offs, scheduling must be implemented carefully. When user-threading parallelism drops off, idle kernel-level threads should be put to sleep at appropriatetimes to avoid wasting CPU resources. Furthermore, it is crucial that the scheduling system provides fairness while preventingany single user thread from monopolizing a kernel thread; otherwise, other user threads may experience short/long term starvation or kernel threads can deadlock waiting for eventsto occur on busy kernel threads. Our model aims to improve RAID performance by identifying the most suitable RAID level for different workloads. We developed a RAID prediction model based on analysis using various machine learning algorithms including Random Forest, XGBoost, Decision Tree and KNN.In conclusion, our research proposes an innovative solution that utilizes machine learning algorithms as part of completely fair scheduler algorithms which can optimize I/O performanceby selecting optimal RAIDs suited towards specific workloads resulting in improved overall system performance.

I. INTRODUCTION

RAID is a data storage virtualization technology that combines multiple physical disk drives into a single logical unit to improve performance, reliability, and data redundancy [1]. The I/O isolation scheme is a proposed method to enhance the performance of RAID systems by efficiently managing data distribution and access across multiple SSDs. This study aims to investigate the effectiveness of the I/O isolation scheme and its impact on RAID performance [2]. The I/O isolation scheme aims to separate I/O requests based on their characteristics and distribute them across multiple SSDs to reduce contention and improve performance. This is achieved through file classification and separation, deployment of files on multiple SSDs, and machine learning techniques for RAID performance measurement. Modern storage systems, such as GFS and Windows Azure, host their data in many storage devices. As the number of data disks increases, so does the probability of data loss or damage due to various disk failures [3].

Therefore, it is crucial to implement RAID systems that can tolerate disk failures and recover lost information while maintaining optimal performance and capacity. In this article, we will examine the standard RAID levels of RAID 0, 5, 6, and 10 to compare their performance differences [4]. The performance of the RAID level prediction model is evaluated using various performance metrics, such as accuracy, precision, recall, F1 score, and confusion matrix. These metrics help assess the effectiveness of the model in predicting the optimal RAID level for different workloads. Most RAID-6 codes cannot provide good load balancing.

Though many dynamic load balancing approaches and static rotating the mappings from logic disks to physical disks [5] stripe by stripe like RAID-5 may alleviate this problem in some level, they either have additional overheads for monitoring the status of disks or also suffer from unbalanced I/ O since the access frequencies of each stripe are different. The I/O cost, which indicates the total I/O accesses among all disks, is another important performance metric in storage systems, because high I/O cost may increase disks' I/O burden and degrade the performance. However, most well-balanced codes suffer from high I/O cost on partial stripe writes (i.e., write to a series of continuous data elements). Different RAID levels necessitate extensive processing for write operations, with varying amounts of computation required for each operation. This added processing introduces latency but does not affect throughput [6]. The latency will depend on the RAID level implementation and the system's processing capability. Hardware RAID typically uses a general-purpose

https://doi.org/10.1049/icp.2024.0907

CPU (often a Power or ARM RISC processor) [7] or a custom ASIC for processing, while software RAID relies on the server's CPU. Although server CPUs are generally faster, they consume system resources. Parity RAID requires data on the disks that is not useful during a healthy read operation and cannot be used to enhance performance. As a result, parity RAID is slightly slower, but this impact is minimal and often not measured, so it can be disregarded. Other factors, such as stripe size, can also influence performance. The comparative study is done in various phases of Standard RAID Levels and Nested RAID architectures like RAID 1+0, RAID 3+5 and RAID 6+0. Observations are extracted from the definition of its architecture [8]. The results are then tabulated and a conclusion about each RAID type is made. RAID configurations are employed mainly in places where we have network-attached storage environments, like places where we need massive storage, reinforcement, or high-end media streaming options. The above model can be used by organizations and individuals to choose the most efficient RAID configuration for their system.

II. TYPES OF CLOUD BASED RAID MODELS

Cloud-based RAID systems have become increasingly popular due to their scalability, reliability, and cost-effectiveness [9]. However, as the number of users and applications using these systems increases, the performance of the underlying storage infrastructure becomes a critical concern. One of the primary bottlenecks in these systems is the I/O subsystem, which can suffer from contention and interference between different workloads. In this paper, we propose an I/O Isolation Scheme to address these issues and improve overall performance in cloud-based RAID systems using Kernel bypass algorithm. Kernel bypass is a technique used to improve the performance of load balancers by bypassing the operating system kernel and directly accessing the network interfacecard (NIC) to read and write packets [10]. This technique can significantly reduce the overhead associated with packet processing in the kernel and improve the throughput and latency of the load balancer.

One example of a load balancer that uses kernel bypass is Maglev, a software network load balancer developed by Google Maglev [11] uses a consistent hash scheduler to distribute requests to a pool of real servers based on a 5-tuple of information from each packet. Maglev can run in kernel bypass mode, which allows it to achieve high throughput and low latency by directly accessing the NIC to read and write packets. When implementing a load balancer scheduler, there are several algorithms that can be used to distribute requests to a pool of real servers.

A. Cloud-RAID

Cloud-RAID (Redundant Array of Independent Disks) is a data storage technology that combines multiple cloud storage services into a single, unified storage system. The main goal of Cloud-RAID is to provide a highly reliable, scalable, and cost-effective storage solution that can meet the growing demands of today's data-driven world. Cloud-RAID works by distributing data across multiple cloud storage providers, using different RAID levels to ensure data redundancy and fault tolerance. This means that even if one of the cloud providers experiences an outage or data loss, the user's data remains safe and accessible through other providers in the Cloud-RAID system. Some of the key benefits of Cloud-RAID include increased storage capacity, improved data availability, and reduced storage costs. By leveraging the power of multiple cloud providers, users can store large amounts of data without worrying about the limitations of a single provider. Moreover, Cloud-RAID offers a flexible and scalable storage solution, allowing users to easily adjust their storage requirements as needed.

B. Uni4Cloud

Uni4Cloud is a comprehensive cloud management platform designed to simplify the deployment, management, and monitoring of cloud resources across multiple providers. The platform aims to provide a unified interface for managing diverse cloud environments, enabling organizations to optimize their cloud investments and streamline their IT operations. Uni4Cloud supports a wide range of cloud providers, including Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP), and many others. The platform offers a variety of features, such as infrastructure provisioning, resource monitoring, cost optimization, and security management, to help organizations effectively manage their cloud resources. With Uni4Cloud, organizations can gain better visibility into their cloud infrastructure, automate routine tasks, and ensure that their cloud resources are running efficiently and securely. The platform also supports integration with popular DevOps tools and processes, making it an ideal solution for organizations looking to embrace modern cloud- native development practices.

C. RAIN (Redundant Array of Independent Nodes)

RAIN is a distributed storage technology that aims to improve data reliability, availability, and performance by distributing data across multiple independent nodes. RAIN is similar in concept to RAID [12], but instead of using multiple disks within a single storage system, RAIN utilizes multiple nodes, each with its own storage and processing capabilities. In a RAIN-based storage system, data is divided into chunks and distributed across the nodes in the array.

Each node is responsible for storing and processing its own data, and the system uses redundancy and fault-tolerance techniques to ensure that data remains available even in the event of node failures. Some of the key benefits of RAIN include improved data availability, increased storage capacity, and enhanced performance. By distributing data across multiple nodes, RAIN can provide better data protection and fault tolerance compared to traditional storage systems. Addition- ally, RAIN-based systems can scale easily by adding more nodes to the array, allowing organizations to accommodate their growing storage needs.

D. RAIC (Redundant Array of Independent Clouds)

RAIC is an advanced cloud storage technology that combines the features of RAID and RAIN to provide a highly reliable, scalable, and secure storage solution. RAIC works by distributing data across multiple independent cloud providers, ensuring that users can access their data even if one of the providers experiences an outage or data loss. In a RAIC-based storage system, data is divided into chunks and distributed across the cloud providers in the array. Each provider is responsible for storing and processing its own data, and the system uses redundancy and fault-tolerance techniques to ensure data availability and reliability. Some of the key benefits of RAIC include increased data availability, improved data protection, and reduced storage costs. By leveraging the power of multiple cloud providers, users can store large amounts of data without worrying about the limitations of a single provider. Moreover, RAIC offers a flexible and scalable storage solution, allowing users to easily adjust their storage requirements as needed.

III. LOAD BALANCING AND SCHEDULING ALGORITHMS

Some examples of load balancing algorithms include:

- Round Robin: Distributes each request sequentially around the pool of real servers. All the real servers are treated as equals without regard to their current load or capacity.
- Static Hash: Distributes requests to the pool of real servers by looking up the destination IP in a static hash table. This algorithm is designed for use in a proxy-cache server cluster.
- Source Hash: Distributes requests to the pool of real servers by looking up the source IP in a static hash table. This algorithm is designed for LVS routers with multiple upstream routers
- Consistent Hash: Distributes requests to the pool of real servers based on a hash of the request URL or other request attributes. This algorithm is designed to minimize the number of requests that are redirected to a different server when a server is added or removed from the pool.

- Inductive-bias-driven Reinforcement Learning (IRL) algorithm: One example of machine learning for load balancing in the Linux kernel is the Inductive-bias-driven Reinforcement Learning (IRL) algorithm. The IRL algorithm uses reinforcement learning to optimize the scheduling of processes based on their resource requirements and system load, with the goal of maximizing system throughput and minimizing response time. The choice of load balancing algorithm depends on the specific requirements of the application and the characteristics of the pool of real servers. For example, if the real servers have different capacities or loads, a dynamic load balancing algorithm may be more appropriate than a static algorithm machine learning can also be used to improve load balancing in the Linux kernel by optimizing the scheduling of processes based on their resource requirements and system load. The Linux kernel's Completely Fair Scheduler (CFS) scheduler tracks process loads by average CPU utilization to balance workload between processor cores. However, this approach overlooks the contention for lower-level hardware resources, which can hinder execution performance in servers running compute-intensive workloads. By using machine learning to optimize the scheduling of processes based on their resource requirements and system load, load balancing in the Linux kernel can be improved to achieve better performance and resource utilization. Troodon: This is a machine-learning based load- balancing application scheduler for CPU-GPU system proposed by Yasir N. Khalid, Muhammad Aleem, Usman Ahmed, Muhammad A. Islam, and Muhammad A. Iqbal [13]. Troodon uses machine learning to optimize the scheduling of workloads onto heterogeneous processors (e.g., CPUs, GPUs, FPGAs) to achieve better performance and resource utilization. Heuristics-based algorithms: Linux achieves load- balancing by regularly monitoring the system state and using some heuristic on the load, currently CPU time used in the last millisecond [14]. These heuristics-based algorithms can be used to optimize the scheduling of processes based on their resource requirements and system load.

A. I/O Isolation for Load Balancing workloads

I/O isolation is the process of segregating the I/O workloads of different applications and users to prevent contention and interference. In cloud-based RAID systems, this can be achieved by implementing several techniques, including:

- Partitioning: Dividing the storage space into separate partitions, each dedicated to a specific application or user.
- Quality of Service (QoS) policies: Defining and enforcing QoS policies to allocate and manage resources according to the specific requirements of each workload [15].
- I/O scheduling: Implementing intelligent I/O scheduling algorithms to prioritize and manage requests from different workloads effectively.

B. Benefits of I/O Isolation for Improved Performance

Implementing I/O isolation in cloud-based RAID systems can offer numerous benefits for improved performance, including:

- Reduced contention: By segregating workloads, I/O isolation can prevent contention between different applications and users, ensuring that each workload receives the resources it requires.
- Enhanced predictability: I/O isolation can improve the predictability of system performance by reducing the impact of interference and contention between workloads.

- Improved resource utilization: Through effective resourceallocation and management, I/O isolation can ensure that storage resources are utilized optimally, leading to better overall system performance.
- Increased user satisfaction: By ensuring that each work- load receives the resources it requires, I/O isolation can lead to improved user satisfaction and reduced complaintsrelated to system performance.

C. Proposed I/O Isolation Scheme and Load Scheduling

Our proposed I/O Isolation Scheme involves the following steps:

- a. Analyzing workload characteristics: We first analyze the I/O workloads of different applications and users to understand their specific requirements and resource usage patterns.
- b. Implementing partitioning and QoS policies: Based on the analysis, we implement partitioning and QoS policies to segregate workloads and allocate resources effectively.
- c. Developing an I/O scheduling algorithm: We design an intelligent I/O scheduling algorithm that can prioritize and manage requests from different workloads based on their specific requirements and resource usage patterns.
- d. Evaluating the scheme: Machine learning used to optimize the scheduling of processes based on their resource requirements and system load to achieve better performance and resource utilization.

The proposed strategy for load scheduling is based on two key factors: delay prediction and lifetime prediction. For each read and write request, the system linearly predicts the execution delay, which helps in estimating the total latency of each SSD. Additionally, the system dynamically predicts the lifetime of each SSD based on its current load. Using these predictions, a novel delay-life-aware load scheduling strategy has been developed. This strategy allocates an appropriate SSD for every incoming write request by considering both latency and lifetime factors associated with individual SSDs. A write distribution algorithm is employed to make this allocation decision. By using this approach, we can optimize resource utilization by allocating writes to less-utilized drives while still maintaining performance requirements through accurate predictions of delays and lifetimes. The result is a more efficient use of resources that minimizes wear-and-tear on individual drives while still providing high levels of service quality to end-users. Overall, this approach provides an effective way to manage storage in cloud environments where data access patterns are unpredictable and workloads can be highly dynamic over time. By considering both performance considerations (in terms of latency) as well as longevity concerns (in terms of drive lifespan), we can ensure optimal use of resources while minimizing downtime or other disruptions caused by hardware failures or degraded performance levels due to aging components. The given terms are important to understand the evaluation metrics: True Positives: The instances when we predicted YES and the actual output was YES. True Negatives: The instances when we predicted NO and the actual output was NO. False Positives: The instances when we predicted YES and the actual output was NO. False Negatives: The instances when we predicted NO and the actual output was YES. approach is to use a content delivery network (CDN) as the user-facing interface and leverage its functions to load balance across your Balancer hosts. The main idea behind the completely fair scheduler (CFS) is to maintain balance (fairness) in providing processor time to tasks. This means processes should be given a fair share of CPU time for execution. The CFS handles CPU resource allocation for executing processes and aims to maximize overall CPU utilization while also maximizing interactive performance. . Real- time processes have higher priority than normal processes. The CFS implementation is based on per-CPU run queues, whose nodes are time-ordered schedulable entities that are kept sorted by red-black trees. The CFS does away with the old notion of per-priorities fixed time-slices and instead aims at giving a fair share of CPU time to task. CFS takes a quite different approach by letting the vruntime facts speak mostly for themselves, which happens to support sleeper fairness. An interactive task, by its very nature, tends to sleep a lotin the sense that it awaits user inputs and so becomes I/O- bound. In summary, the Completely Fair Scheduler (CFS) is a process scheduler that is part of the Linux kernel and aims to provide fairness in providing processor time to tasks. It handlesCPU resource allocation for executing processes and aims to maximize overall CPU utilization while also maximizing interactive performance. The CFS implementation is based on per-CPU run queues and red-black trees, and it does away with the old notion of per-priorities fixed time-slices. The CFS also handles I/O-bound tasks by letting the vruntime facts speak mostly for themselves, which supports sleeper fairness.

IV. I/O Isolation in a Completely Fair Scheduler Algorithm using the appropriate model

Several studies have been conducted in the past to address the issue of I/O contention and interference in cloud-based RAID systems. Some of these studies have focused on the development of intelligent I/O scheduling algorithms, while others have proposed the use of partitioning and QoS policies for effective resource allocation and management. Our proposed I/O Isolation Scheme builds upon these previous works and provides a comprehensive solution for improved performance in cloud-based RAID systems. The following arethe steps involved in I/O Isolation

- The RAID data-set is generated from survey sent to various tech staff in the organization
- The data-set attributes are divided into input and output classes.
- One Hot Encoding technique is used for converting categorical, textual input data into numerical data. The output variable “RAID level” is mapped using label encoding.
- This data is fed to the proposed RAID prediction model for training the dataset.
- The proposed model is trained and tested for various classification-based machine learning algorithms and the

accuracy score is thus calculated. We run the fair scheduler algorithm.

Task Selection →

- Initialize the system with per-CPU run queues and red- black trees. For each incoming task, perform the follow- ing steps-
  - Determine the task's priority (real-time or normal).
  - Calculate the task's vruntime based on its priority, previous vruntime, and the amount of time it has spent sleeping.
  - Insert the task into the red-black tree based on its vruntime.
  - Update the per-CPU run queue with the new task.
  - Initialize the priority queue PQ, read-write workload RW, and the counter C for each stripe.
  - For each of the 2000 randomly generated 3-tuple ¡S, L, T¿, do the following-
    - Calculate the priority P of the current tuple using the mathematical equation: $P = W(S, L, T) = \alpha S + \beta L + \lambda T$ where α, β, and λ are the weights assigned to each element of the tuple.
    - Insert the tuple ¡S, L, T, P¿ into the priority queue PQ, sorted in descending order of priority P. While the
    - priority queue PQ is not empty, do the following:
      - Dequeue the tuple with the highest priority from PQ.
      - Perform the read/write operation specified by the dequeued tuple.
      - Update the counter C for the corresponding stripe.
    - Once the priority queue PQ is empty, the read- write workload RW has been processed, and the I/O isolation has been achieved. Mathematical Equation:

$$\text{Priority} P = W(S, L, T) = \alpha \quad S * \beta \quad L + \lambda \quad T \quad * \tag{1}$$

      where: - S is the stripe element - L is the range of data elements (1 to 20) - T is the time taken for the operation - α, β, and λ are the weights assigned to each element of the tuple - P is the priority of the tuple
    - While there are tasks to be executed, perform the following steps:
      - Select the task with the lowest vruntime from the red-black tree.
      - Execute the selected task.
      - Update the task's vruntime based on its execu- tion time.
      - If the task has not completed execution, re-insert it into the red-black tree based on its updated vruntime.
      - Update the per-CPU run queue with the exe- cuted task.
      - Continue executing tasks until all tasks are completed.
      - If new tasks arrive, go back to step 2.

V. Machine Learning in LOAD Scheduling algorithms using classification models

RAID technology and levels have been instrumental in improving data storage reliability and performance, with various RAID configurations offering different benefits. In the context of cloud storage systems, I/O performance is a critical factor that determines the overall efficiency and user experience. Research initiatives are exploring machine learning techniques to predict RAID performance and optimize cloud storage systems. By leveraging machine learning algorithms, it is possible to analyze and predict the performance of various RAID configurations under different workloads, leading to more informed decisions when designing and managing cloud storage systems [16]. Existing cloud-of-clouds research initiatives aim to enhance the resilience and performance of cloud storage systems by distributing data across multiple cloud providers. This approach can help mitigate risks associated with single-provider outages and improve overall reliability. However, there is a gap in understanding the optimal RAID configurations and machine learning techniques to maximize the benefits of cloud-of-clouds storage systems. Further research and development in this area can lead to emerging insights and new ideas that can revolutionize the way data is stored and managed in the cloud, ensuring greater reliability, performance, and cost-efficiency for businesses and users alike.. The ML algorithms cannot process the categorical values in the dataset. This is the reason behind using the One hot encoding technique which converts the categorical values in the dataset into numerical values suitable for processing by the ML algorithms. In the next step, the RAID dataset is trained with classification models like Logistic Regression, Random Forest, XGBoost, KNN, Decision Tree, Naıve Bayes, and SVM. The performance of the ML models is compared by using metrics such as accuracy, precision, recall, and F1-score. The most accurate algorithm is used for prediction and linked to a Web-Application for real-time user deployment. If two load balancers remain in active-standby mode, only one load balancer [17] handles the incoming requests at any given time, while the other remains on standby. In scenarios where incoming requests require prioritization based on the type of operation, utilizing machine learning algorithms can be a suitable solution to intelligently distribute the traffic. To further improve scalability and fault tolerance, a cluster of load balancers can be implemented with the same domain name, allowing DNS to perform round-robin load balancing. This approach enables auto-scaling of load balancers by dynamically adding or removing records in the DNS server/s, providing a more resilient and efficient load balancing solution. Most of the RAID levels render good protection against data errors or hardware failures but they do not provide any protection from any data loss due to hazards (fire, waters) or soft errors such as user error, software malfunction. In order to

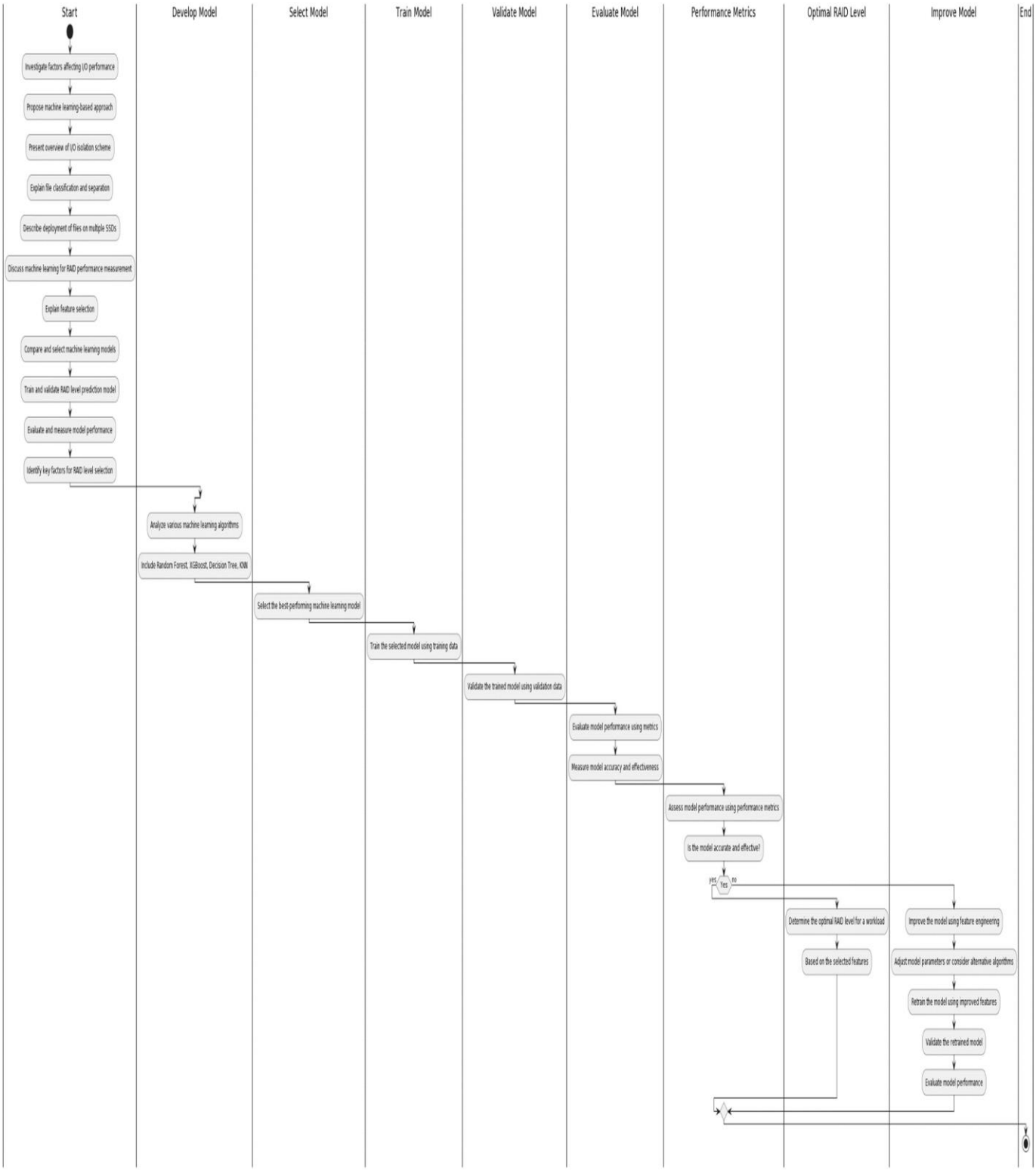

Fig. 1. Diagram showing the sequence of the work distribution for a machine learning model.

overcome the shortcomings in existing codes discussed above, the authors should design a new code with properties of good load balancing, low I/O cost, and good performance on both normal reads and degraded reads. In this work, we have started by contemplating different research papers and articles to comprehend and investigate diverse RAID levels. The RAID Level Prediction Model Training and Validation process involves selecting appropriate training and validation

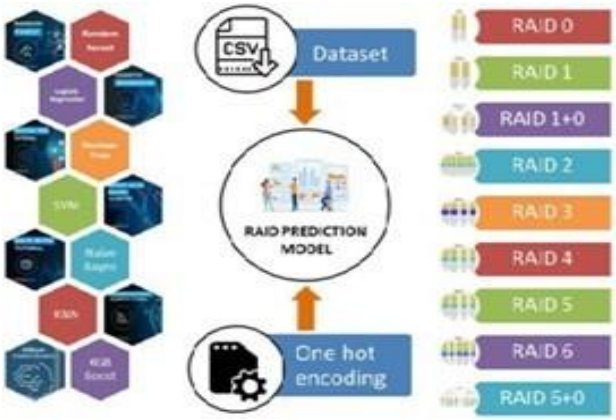

Fig. 2. Machine Learning for RAID Prediction Model.

data-sets, ensuring the model is well-trained and can generalize well to new data. Cross-validation and hyper-parameter tuning techniques are employed to optimize the model's performance and prevent over-fitting. Model selection and comparison involve evaluating the performance of different machine learning models and selecting the best one based on metrics such as accuracy, precision, recall, and F1-score. This comprehensive approach to RAID performance measurement and prediction using machine learning ensures efficient and reliable RAID system management, ultimately leading to improved storage performance and data protection. Machine learning algorithms such as Inductive-bias-driven Reinforcement Learning (IRL) can be used to optimize the scheduling of processes based on their resource requirements and system load, with the goal of maximizing system throughput and minimizing response time. Adaptive load balancing based on machine learning can be used for iterative parallel applications to optimize the distribution of workloads among processors based on their performance characteristics. Paralleling machine learning algorithms require synchronization protocols to ensure model synchronization among different model replicas and partitions. We use a Completely Fair Scheduler (CFS) and compare with learning algorithms like Logistic Regression, Random Forest, and Decision Tree. model was executed using Random Forest and Decision Tree algorithm. Evaluation metrics like accuracy, recall score, precision score, and F1 score are used to analyze the results and select the most efficient algorithm to predict the RAID level in the WebApp interface. Overall, on evaluating various machine learning algorithms using the above 4 metrics, it is observed that the Random Forest model outperformed and is more efficient and accurate in predicting the output class correctly. Therefore, a single page web application is created which contains a form. This form consists of 6 questions to fetch the user requirements and pass it to the RAID predicting machine learning model. The requirements are then pre-processed into a format for which the model is trained. Random forest algorithm is then used to predict the output class. In summary, supervised machine learning algorithms such as SVM and decision trees can be used for classification and regression tasks in load balancing, while unsupervised machine learning algorithms such as K-means clustering and PCA can be used for clustering and dimensionality reduction. Reinforcement learning algorithms such as IRL can be used to optimize the scheduling of processes based on their resource requirements and system load.

TABLE I
EXECUTION OF THE DIFFERENT MACHINE LEARNING ALGORITHMS

Classifier	Accuracy	Recall	Precision	F1 Score
Logistic Regression	96.37	96.37	96.5	96.35
Random Forest	97.65	97.65	97.70	97.64
Decision Tree	96.79	96.79	97.08	96.8
Naive Bayes	77.14	77.14	83.23	75.96
KNN	93.16	93.16	93.82	93.08
SVM	91.45	91.43	92.29	91.39

VI. RESULTS AND DISCUSSION

Measurement involves the use of advanced algorithms and techniques to analyze and predict the performance ofRAID (Redundant Array of Independent Disks) systems. Key feature points include feature selection and extraction for RAID performance analysis, which helps in identifying the most significant variables affecting the performance of RAID systems. The load scheduler algorithm using the Completely Fair Scheduler (CFS) aims to provide fairness in allocating processor time to tasks. It handles CPU resource allocation for executing processes and aims to maximize overall CPU utilization while also maximizing interactive performance. The CFS implementation is based on per-CPU run queues andred-black trees, and it does away with the old notion of per-priorities fixed time-slices. The CFS also handles I/O-bound tasks by letting the run-time facts speak mostly for themselves, which supports sleeper fairness. In summary, machine learning can be used in I/O isolation and load balancing of parallel threads to optimize the performance of multiprocessor computer systems. Load balancing algorithms such as IRL can be used to optimize the scheduling of processes based on their resource requirements and system load, while adaptive load balancing based on machine learning can be used for iterative parallel applications. Synchronization protocols are also required for parallelizing machine learning algorithms to ensure model synchronization.

VII. CONCLUSIONS

The proposed load scheduling strategy utilizes delay prediction and lifetime prediction to optimize resource utilization and maintain performance requirements. The priority queue PQ is initialized, along with the read-write workload RW and a counter C for each stripe. The system processes 2000 randomly generated 3-tuples ¡S, L, T¿ by calculating their priority P and sorting them in the priority queue. As the priority queue is processed, read/write operations are performed, and the counter for each stripe is updated. Once the priority queue is empty, the read-write workload has been processed, andI/O isolation is achieved. This approach allows for efficient allocation of writes to less-utilized drives and accurate predictions of delays and lifetimes, ensuring optimal use of resources while minimizing downtime.

By considering both performance (latency) and longevity (drive lifespan), the strategy ensures a well-balanced and efficient load scheduling system. By conducting a comprehensive study on various factors affecting I/O performance in cloud-based RAID systems, this paperaims to provide valuable insights into optimizing RAID level selection using machine learning techniques. The proposed approach not only helps in improving I/O performance but also contributes to the efficient utilization of storage resources in cloud environments.

REFERENCES

[1] D. A. Patterson, P. Chen, G. Gibson, and R. H. Katz, "Introduction to redundant arrays of inexpensive disks (raid)," in COMPCON Spring 89, pp. 112–113, IEEE Computer Society, 1989.

[2] Y. Zheng, X. Lu, M. Zhang, S. Chen, Y. Zheng, X. Lu, M. Zhang,and S. Chen, "Biogeography-based optimization in machine learn- ing," Biogeography-Based Optimization: Algorithms and Applications, pp. 199–217, 2019.

[3] D. Saxena, J. Kumar, A. K. Singh, and S. Schmid, "Performance analysisof machine learning centered workload prediction models for cloud," IEEE Transactions on Parallel and Distributed Systems, vol. 34, no. 4, pp. 1313–1330, 2023.

[4] J. Jiang, L. Yu, J. Jiang, Y. Liu, and B. Cui, "Angel: a new large-scale machine learning system," National Science Review, vol. 5, no. 2, pp. 216–236, 2018.

[5] S. Boag, P. Dube, B. Herta, W. Hummer, V. Ishakian, K. Jayaram, M. Kalantar, V. Muthusamy, P. Nagpurkar, and F. Rosenberg, "Scalablemulti-framework multi-tenant lifecycle management of deep learning training jobs," in Workshop on ML Systems, NIPS, 2017.

[6] S. Sahana, R. Bose, and D. Sarddar, "Harnessing raid mechanism for enhancement of data storage and security on cloud," Brazilian Journalof Science and Technology, vol. 3, pp. 1–13, 2016.

[7] Q. Liu and L. Xing, "Survivability and vulnerability analysis of cloud raid systems under disk faults and attacks," International Journal of Mathematical, Engineering and Management Sciences, vol. 6, no. 1,p. 15, 2021.

[8] M. Kumar, C. C. Columbus, E. B. George, and T. A. B. Raj, "A virtual cloud storage architecture for enhanced data security," COMPUTER SYSTEMS SCIENCE AND ENGINEERING, vol. 44, no. 2, pp. 1735–1747, 2023.

[9] N. Nayak, S. Mishra, S. Chowdhury, P. Dutta, and G. Ramya, "Rfd based technique on qos parameters using cloud computing," in 6th Smart Cities Symposium (SCS 2022), vol. 2022, pp. 227–231, 2022.

[10] J. Chen, S. S. Banerjee, Z. T. Kalbarczyk, and R. K. Iyer, "Machine learning for load balancing in the linux kernel," in Proceedings of the 11th ACM SIGOPS Asia-Pacific Workshop on Systems, pp. 67–74, 2020.

[11] D. E. Eisenbud, C. Yi, C. Contavalli, C. Smith, R. Kononov, E. Mann-Hielscher, A. Cilingiroglu, B. Cheyney, W. Shang, and J. D. Hosein, "Maglev: A fast and reliable software network load balancer," in 13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16), pp. 523–535, 2016.

[12] J. Saravanan, T. Ananth kumar, P. Divya, R. Partipan, R. Partipan,and P. K. Dutta, "Iot based advanced prediction methodology for fault localization in machine monitoring," in 6th Smart Cities Symposium (SCS 2022), vol. 2022, pp. 253–257, 2022.

[13] Y. N. Khalid, M. Aleem, U. Ahmed, M. A. Islam, and M. A. Iqbal, "Troodon: A machine-learning based load-balancing application scheduler for cpugpu system," Journal of Parallel and Distributed Computing, vol. 132, pp. 79–94, 2019.

[14] P. K. Dutta, O. Mishra, and M. Naskar, "Improving situational awareness for precursory data classification using attribute rough set reduction ap-proach," International Journal of Information Technology and Computer Science, vol. 5, no. 12, pp. 47–55, 2013.

[15] A. Matuka, S. S. Asafo, G. O. Eweke, P. Mishra, S. Ray, M. Abotaleb, T. Makarovskikh, A. Alhussan, A. Abdelhamid, E. El-Kenawy, P. Dutta,and S. Chowdhury, "Analysing the impact of covid-19 outbreak and eco-nomic policy uncertainty on stock markets in major affected economies," in 6th Smart Cities Symposium (SCS 2022), vol. 2022, pp. 372–378, 2022.

[16] S. Banerjee, S. Jha, Z. Kalbarczyk, and R. Iyer, "Inductive-bias-driven reinforcement learning for efficient schedules in heterogeneous clusters,"in International Conference on Machine Learning, pp. 629–641, PMLR,2020.

[17] P. W. Poteete, "Organically distributed sustainable storage clusters," Array, p. 100275, 2023.